

BÁO CÁO KỸ THUẬT: SỬA LỖI HARDCODE URL VÀ NGHẼN CỔ CHAI TRONG RESTTEMPLATE

Dịch vụ: **Order-Service (VietMart)** | Cấp độ: **Vận dụng cơ bản** | Trạng thái: **HOÀN THÀNH 100%**

1. TỔNG QUAN BỐI CẢNH & HIỆN TRẠNG HỆ THỐNG

Trong kiến trúc Microservices của nền tảng thương mại điện tử **VietMart**, **Order Service** cần giao tiếp liên dịch vụ (inter-service communication) với **Product Service** để truy vấn thông tin chi tiết tên và đơn giá của sản phẩm khi hiển thị chi tiết đơn hàng. Hệ thống sử dụng **Netflix Eureka** làm Service Discovery để quản lý động các service instances.

Một thực tập sinh đã cài đặt lớp **ProductServiceClientRT** bằng việc tạo mới đối tượng **RestTemplate** thủ công và hardcode địa chỉ IP máy cá nhân. Mặc dù chạy thử nghiệm thành công ở môi trường Local Development, ứng dụng đã lập tức sập khi triển khai lên môi trường Staging/Production.

2. PHÂN TÍCH CHI TIẾT 3 LỖI KỸ THUẬT & HẬU QUẢ PRODUCTION

LỖI 1: Khởi tạo trực tiếp RestTemplate không có @LoadBalanced

Hiện trạng code lỗi:

```
private final RestTemplate restTemplate = new RestTemplate();
```

Phân tích nguyên nhân:

- Khởi tạo thủ công bằng từ khóa **new** vi phạm nguyên lý Dependency Injection (DI) và Inversion of Control (IoC) của Spring Framework.
- Thiếu annotation **@LoadBalanced**, dẫn đến Spring Cloud không thể đính kèm bộ đánh chặn **LoadBalancerInterceptor** vào RestTemplate.

Hậu quả trong môi trường Production:

- Không phân giải được Service Name:** Khi client gọi đến logical URI **http://product-service/...**, RestTemplate thông thường sẽ coi **product-service** là một DNS hostname mạng công cộng và lập tức ném ra ngoại lệ **UnknownHostException**.
- Vô hiệu hóa Service Discovery:** Không tận dụng được danh sách các IP/port động mà Eureka Server đã đăng ký.
- Không có Client-Side Load Balancing:** Khi Product Service có nhiều instances (scale out 3-10 nodes), Order Service hoàn toàn không thể cân bằng tải (Round Robin), dẫn đến tải không đồng đều hoặc sập hệ thống.

LỖI 2: Hardcode địa chỉ IP và Port vật lý ("http://192.168.1.45:8082")

Hiện trạng code lỗi:

```
String url = "http://192.168.1.45:8082/api/products/{id}";
```

Phân tích nguyên nhân:

- Liên kết chặt chẽ (tight coupling) tăng logic ứng dụng với vị trí topo mạng vật lý của một máy cụ thể.

Hậu quả trong môi trường Production:

- Sập kết nối ngay khi đổi môi trường:** Khi đóng gói Container Docker/Kubernetes hay cloud VM, địa chỉ IP thay đổi liên tục theo từng chu kỳ vòng đời. Khi deploy lên Staging/Production, gọi đến `192.168.1.45` sẽ sinh lỗi `ConnectException: Connection refused`.
- Tạo điểm nghẽn duy nhất (Single Point of Failure):** Mọi request từ toàn bộ hệ thống Order Service đều dồn vào đúng một node 192.168.1.45. Nếu instance này chết, toàn bộ tính năng tra cứu đơn hàng ngừng hoạt động 100%.

LỖI 3: Không cấu hình Timeout (connectTimeout & readTimeout)

Hiện trạng code lỗi:

```
return restTemplate.getForObject(url, ProductInfo.class, productId);
```

Phân tích nguyên nhân:

- `RestTemplate` mặc định trong Spring Boot/Java sử dụng giá trị timeout bằng `-1` (vô hạn) hoặc phụ thuộc vào TCP stack của hệ điều hành (thường kéo dài từ vài phút đến vài chục phút).

Hậu quả trong môi trường Production:

- Thread Starvation (Cạn kiệt Thread Pool):** Khi Product Service bị treo, đầy RAM, lock Database hoặc mạng trễ cao, các worker thread của Tomcat (thường giới hạn 200 threads) ở Order Service sẽ bị block mãi mãi để chờ phản hồi.
- Hiệu ứng sụp đổ dây chuyền (Cascading Failure):** Khi toàn bộ 200 threads bị chiếm dụng, tất cả các API khác của Order Service (tạo đơn, hủy đơn, xem lịch sử) đều bị từ chối phục vụ (503 Service Unavailable). Một lỗi cục bộ ở Product Service đã kéo sập toàn bộ Order Service.
- Trải nghiệm người dùng tồi tệ:** Người dùng phải chờ đợi màn hình loading vô tận trước khi bị timeout báo lỗi không rõ nguyên nhân.

3. GIẢI PHÁP & THIẾT KẾ KIẾN TRÚC MỚI ĐẠT CHUẨN

CÁC CẢI TIẾN ĐÃ THỰC HIỆN

- Tách Configuration Bean riêng biệt:** Tạo lớp `RestTemplateConfig` cung cấp Spring Bean `RestTemplate` được gắn `@LoadBalanced`.
- Cấu hình Timeout chặt chẽ:** Sử dụng `SimpleClientHttpRequestFactory` thiết lập `connectTimeout = 2 giây` và `readTimeout = 3 giây`.
- Chuyển sang định danh Logical Service ID:** Sử dụng URI `http://product-service/api/products/{id}` để Eureka Server tự động định tuyến và phân phối tải.
- Áp dụng Fault Tolerance & Fallback Pattern:**
 - Bắt ngoại lệ `ResourceAccessException` (xảy ra khi mạng timeout hoặc connection refused) để trả về thông tin sản phẩm mặc định (fallback data), giữ cho hệ thống luôn chạy mượt mà.
 - Bắt ngoại lệ `HttpClientErrorException.NotFound` (HTTP 404) để ném ra `ProductNotFoundException` rõ ràng phục vụ tầng nghiệp vụ.

Mã nguồn RestTemplateConfig.java

```
package com.example.bai1.config;

import org.springframework.cloud.client.loadbalancer.LoadBalanced;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.web.client.RestTemplate;
import java.time.Duration;

@Configuration
public class RestTemplateConfig {

    @Bean
    @LoadBalanced
    public RestTemplate restTemplate() {
        SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
        factory.setConnectTimeout(Duration.ofSeconds(2)); // connectTimeout = 2s
        factory.setReadTimeout(Duration.ofSeconds(3));    // readTimeout = 3s
        return new RestTemplate(factory);
    }
}
```

Mã nguồn ProductServiceClientRT.java (Đã sửa hoàn chỉnh)

```
package com.example.bai1.client;

import com.example.bai1.dto.ProductInfo;
import com.example.bai1.exception.ProductNotFoundException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.client.RestTemplate;

@Component
public class ProductServiceClientRT {

    private static final Logger log = LoggerFactory.getLogger(ProductServiceClientRT.class);
    public static final String PRODUCT_SERVICE_URL = "http://product-service/api/products/{id}";

    private final RestTemplate restTemplate;

    @Autowired
    public ProductServiceClientRT(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    public ProductInfo getById(Long productId) {
        try {
            return restTemplate.getForObject(PRODUCT_SERVICE_URL, ProductInfo.class, productId);
        } catch (HttpClientErrorException.NotFound ex) {
            log.error("Product with id {} not found (HTTP 404): {}", productId, ex.getMessage());
            throw new ProductNotFoundException(productId);
        } catch (ResourceAccessException ex) {
            log.warn("Timeout or network error calling product-service for productId {}: {}. Returnin
                productId, ex.getMessage());
            return getFallbackProduct(productId);
        }
    }

    protected ProductInfo getFallbackProduct(Long productId) {
        ProductInfo fallback = new ProductInfo();
        fallback.setId(productId);
        fallback.setName("Sản phẩm tạm thời không khả dụng");
        fallback.setPrice(0.0);
        fallback.setDescription("Dịch vụ sản phẩm đang gián đoạn hoặc timeout. Vui lòng thử lại sau."
        return fallback;
    }
}
```

4. KẾT QUẢ KIỂM THỬ TỰ ĐỘNG (UNIT TESTS)

Kiểm thử đơn vị được xây dựng bằng **JUnit 5** và **Mockito**, độc lập hoàn toàn với hạ tầng mạng và server ngoài.

STT	Tên Kịch Bản Kiểm Thử (Unit Test Case)	Mục Tiêu & Điều Kiện Kích Hoạt	Thời Gian	Kết Quả
1	testGetById_Success (Happy Path)	Giả lập Product Service phản hồi mã HTTP 200 kèm DTO sản phẩm chuẩn (Dell XPS 15). Kiểm tra trích xuất dữ liệu chính xác.	0.853s	PASSED
2	testGetById_TimeoutFallback (Timeout & Fallback)	Giả lập RestTemplate ném ngoại lệ ResourceAccessException mô phỏng lỗi Timeout quá ngưỡng 3s. Kiểm tra fallback an toàn.	0.007s	PASSED
3	testGetById_NotFoundThrowsException (404 Error)	Giả lập HTTP 404 Not Found từ Product Service. Kiểm tra ném đúng ngoại lệ ProductNotFoundException .	0.049s	PASSED
4	contextLoads (Spring Context Boot)	Kiểm tra tính tương thích và khởi tạo trơn tru toàn bộ Spring Bean trong Application Context.	0.008s	PASSED

Bảng chứng thực thi Gradle Test CLI:

```
> Task :testClasses
> Task :test

ProductServiceClientRTTest > 1. Test gọi thành công (happy path): trả về thông tin sản phẩm chính xác
ProductServiceClientRTTest > 3. Test xử lý lỗi 404 (Not Found): ném ra ProductNotFoundException PASSE
ProductServiceClientRTTest > 2. Test mô phỏng timeout: ResourceAccessException ném ra và trả về fallback
Bai1ApplicationTests > contextLoads() PASSED

BUILD SUCCESSFUL in 18s
4 actionable tasks: 4 executed
```

5. KẾT LUẬN

Dự án đã giải quyết triệt để toàn bộ 3 lỗi kiến trúc cơ bản của hệ thống Microservices trong **Order Service**:

- Loại bỏ hoàn toàn địa chỉ IP tĩnh cứng, thay bằng cơ chế Dynamic Service Discovery qua Eureka.
- Tích hợp Client-side Load Balancing thông qua **@LoadBalanced**.
- Bảo vệ tài nguyên server bằng cấu hình timeout giới hạn thời gian thực thi (2s connect, 3s read).

- Áp dụng mô hình phòng thủ chịu lỗi (Fault Tolerance) với Fallback Mechanism, đảm bảo độ sẵn sàng cao (High Availability) cho hệ sinh thái VietMart.

Báo cáo kết quả bài tập 1 - Session 10 - Hệ thống thương mại điện tử VietMart Microservices